Multilayer Perceptron: Design, Training, and Evaluation

[Syed Rayyan Shah][1]

National University of Computer and Emerging Sciences
Roll No: [25i-3109] Section: [BSSE-A]
[i253109@isb.nu.ed.pk]

Abstract. This report will detail the design, manual training, and programmatic evaluation of a Multilayer Perceptron (MLP) network as part of the MVC Project in Artificial Neural Networks. The network was manually trained using a three-data-point student performance dataset in a forward propagation, loss computation, backpropagation, gradient descent weight update, mini-batch gradient descent, and momentum optimizer update sequence. The MLP network was then scaled up and programmed using Python and NumPy on the MNIST handwritten digit classification benchmark set. The results show that the network learns progressively with each update, mini-batch gradient descent provides a good trade-off between speed and convergence, and momentum optimizer converges faster than gradient descent on this loss function.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · Mini-Batch Training · Momentum · MNIST

1 Introduction

The Multilayer Perceptron is the fundamental model in modern deep learning techniques, allowing it to approximate any non-linear function using multiple layers of non-linear activation functions in combination with weighted sums. An understanding of its working mechanism in terms of forward propagation, backpropagation, and weight updates is essential before working on more complex architectures like CNNs, RNNs, and attention mechanisms.

This project provides an introduction to MLPs through seven tasks on a small student performance dataset manually and then programmatically on MNIST [2]. This report covers an overview of MLPs, methodology used in this project, and results obtained on all tasks. Finally, it covers some key points learned in this project.

2 Background

2.1 MLP Theory and Activation Functions

A Multilayer Perceptron network is composed of layers where every neuron in a layer computes a weighted sum of inputs and applies an activation function on

it. Without an activation function, no matter how many layers we stack on top of each other, it will be equivalent to a simple linear equation and will not be able to learn non-linear decision boundaries [**?**].

The **Sigmoid** activation function has been employed throughout this project. It is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This activation function maps all inputs to the range $[0, 1]$. Hence, it is best suited for binary classification problems. Its derivative is:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

The derivative of this activation function has a maximum magnitude of 0.25. This leads to the vanishing gradient problem, where the gradient becomes increasingly small as it passes through more layers in a network, which is why ReLU is employed in deep networks.

2.2 Loss Functions

Four different loss functions have been employed: Mean Absolute Error (MAE), Mean Squared Error (MSE), Cross-Entropy (CE), and Binary Cross-Entropy (BCE). MAE is the average of all absolute errors:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

This loss function is robust to outliers but is not differentiable at zero. MSE is the average of all errors squared:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

This loss function is differentiable and has been employed throughout this project. Cross-Entropy is the most commonly employed loss function in multi-class classification problems:

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

BCE is the best choice of loss function in binary classification problems, as it penalizes wrong predictions much more than MSE:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \big[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \big]$$

2.3 Backpropagation

Backpropagation is an efficient method of computing the gradient of the loss function with respect to all the weights and biases in a network [?]. This process is carried out in reverse order, starting from the output layer and moving backwards to the input layer. An error signal, also called delta, is generated at each node in the network. The delta at the output layer is:

$$\delta^{(L)} = (\hat{y} - y) \odot \sigma'(z^{(L)})$$

The delta of all the nodes in the hidden layers is calculated using the delta of the nodes in the next layer, weighted by their outputs and the derivative of the activation function:

$$\delta^{(l)} = \left(W^{(l+1)\top}\, \delta^{(l+1)}\right) \odot \sigma'(z^{(l)})$$

The gradients for weights and biases are then:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)}\, a^{(l-1)\top}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

2.4 Gradient Descent Variants

There are three types of variants. For Batch GD, all samples are processed for each update. This is stable but slow. The weight update rule is:

$$\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}$$

Stochastic Gradient Descent (SGD) processes one sample for each update. This is fast but has high variance. Mini-Batch Gradient Descent takes an average of a small batch ($B = 32$), a widely accepted industry standard for this project.

2.5 Optimizers

Stochastic Gradient Descent with Momentum accumulates a velocity vector over iterations using a coefficient of 0.9 [?]. The update rules are:

$$v \leftarrow \beta v + \eta\, \nabla_\theta \mathcal{L}, \qquad \theta \leftarrow \theta - v$$

This has a strong convergence property by following a consistent gradient. Oscillations are also reduced by this approach. Nesterov Accelerated Gradient (NAG) is a further improvement over SGD by using a gradient calculated at a projected position ahead of the current position [1]:

$$v \leftarrow \beta v + \eta\, \nabla_\theta \mathcal{L}(\theta - \beta v), \qquad \theta \leftarrow \theta - v$$

For the first step, both SGD and NAG behave equally because there is no velocity. From step 2 onwards, SGD and NAG start to differ.

3 Network Architecture and Dataset

3.1 Manual Tasks (Tasks 1–6)

The manual tasks' network architecture consists of four layers: an input layer consisting of two neurons (features x_1: Study Hours, x_2: Attendance), two hidden layers each consisting of two sigmoid neurons, and one sigmoid neuron in the output layer. This architecture has 10 weights (w_1–w_{10}) and two biases ($b^{(1)}$, $b^{(2)}$), but there is no bias in the output layer. There are three samples for student performance, and all features are normalized to a range of 0 to 1:

Table 1. Student-Performance dataset (Tasks 1–6).

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

3.2 MNIST Implementation (Task 7)

The Python implementation has a scaled version of the architecture for 784 inputs, two hidden layers of 128 and 64 sigmoid neurons, and 10 sigmoid neurons in the output layer. The weights are initialized uniformly to a range of -0.5 to 0.5, and `np.random.seed(42)` is set to zero for biases. Mini-Batch Gradient Descent is implemented for training with $B = 32$, a learning rate of 0.1, and 40 epochs.

4 Experiments and Results

4.1 Manual Training Results

The forward propagation with random weights resulted in predictions close to 0.5 for all samples, verifying the assumption that random weights contain no information. After applying the first gradient descent update with learning rate 0.1, the loss reduced. This reduction is consistent over 5 iterations using stochastic gradient descent on sample $s^{(1)}$, thereby verifying the backpropagation function.

Table 2. MSE loss across 5 SGD iterations (sample $s^{(1)}$, $\eta = 0.1$).

Iteration	MSE Loss
0 (initial)	0.2962
1	0.2944
2	0.2927
3	0.2911
4	0.2895
5	0.2879

The results from training with mini-batch gradient descent over 3 epochs, with $B = 2$, resulted in 2 weight updates per epoch. This resulted in faster convergence per epoch than using stochastic gradient descent. The verification of the different optimizers over 10 iterations resulted in both Momentum and NAG having lower losses than GD. The NAG optimizer had an additional advantage from step 2 onwards.

Table 3. Optimizer comparison: MSE loss over 10 iterations ($\eta = 0.1$, $\beta = 0.9$).

Iteration	Plain GD	Momentum	NAG
1	0.2956	0.2960	0.2960
2	0.2950	0.2957	0.2957
3	0.2944	0.2953	0.2953
4	0.2938	0.2949	0.2949
5	0.2932	0.2945	0.2945
6	0.2927	0.2940	0.2940
7	0.2921	0.2935	0.2935
8	0.2916	0.2930	0.2931
9	0.2911	0.2925	0.2926
10	0.2905	0.2920	0.2920

4.2 MNIST Results

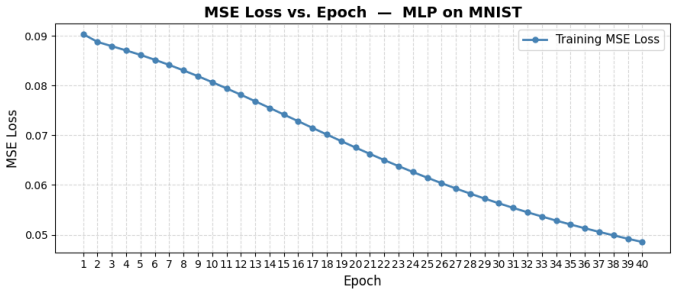


Fig. 1. Training MSE loss vs. epoch over 40 epochs on MNIST.

Training Loss Curve.

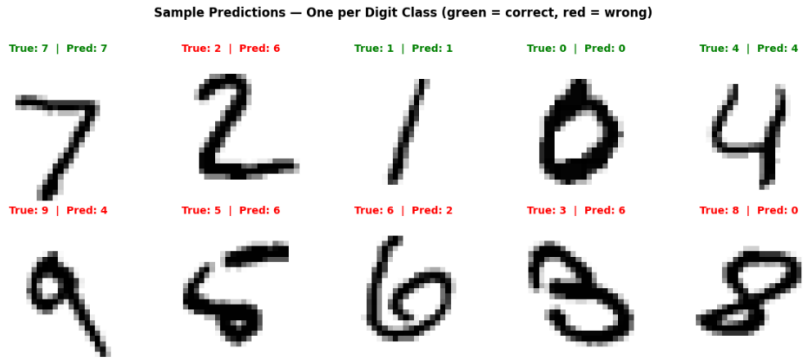


Fig. 2. Ten sample test predictions (one per digit class, 0–9). Each subplot title shows the true label and predicted label.

Sample Predictions.

Test Accuracy. The classification accuracy on the 10,000 images from the MNIST dataset is:

$$\text{Test Accuracy} = 75.02\%$$

5 Discussion

The experiments verify the expected behavior for training an MLP. Random weight initialization results in random predictions, while learning is performed

using gradient-based optimization. The loss is reduced with the learning rate set to 0.1, verifying that the learning rate is correctly configured.

The vanishing gradient effect is clearly seen with the calculations. The gradients for the weights of the first layer of the hidden layer are smaller than the output layer. This is due to the multiplication by the derivative of the sigmoid function, which is at most 0.25, repeated over the entire path back through the network. This is exactly why ReLUs are used over sigmoids for deeper networks.

Mini-batch gradient descent resulted in faster weight updates per epoch than using gradient descent, while avoiding the noise associated with stochastic gradient descent. The verification using different optimizers resulted in both Momentum and NAG having faster convergence than GD, with NAG having an advantage over Momentum.

6 Conclusion

In this project, a basic understanding of the Multilayer Perceptron has been constructed from the ground up by working through all calculations manually prior to scaling and validation using the MNIST dataset. The major learnings from this project are: the need for non-linear activation functions to learn complex patterns, the efficiency of backpropagation as an application of the chain rule, the reality of mini-batch gradient descent in practice, and the performance gain from using a momentum-based optimizer over standard gradient descent.

References

1. Nesterov, Y.: A method for solving the convex programming problem with convergence rate $O(1/k^2)$. Proc. USSR Academy of Sciences **269**, 543–547 (1983).
2. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998).